COMPUTER ORGANIZATION & ARCHITECTURE

MARIE Architecture

From C source to RTL simulation

Project deliverables

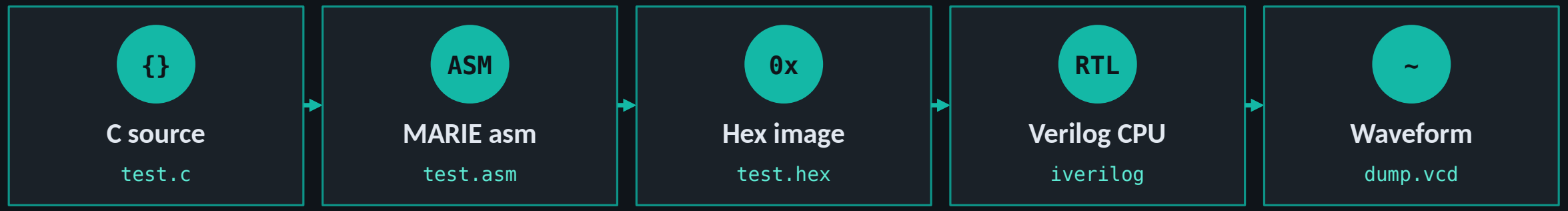
1.1 C → MARIE assembly
1.2 Hex encoding for memory load
1.3 iverilog simulation + waveform

Toolchain

Icarus Verilog · vvp · GTKWave
MARIE ISA: 4-bit op + 12-bit address
16-bit word, 4096-word memory

PROJECT AT A GLANCE

From a 4-line C loop to silicon simulation



20
instructions in code section

5
loop iterations executed

79
instructions retired before HALT

17
memory store transactions

Goal: prove that a hand-written MARIE program runs correctly on the provided Verilog CPU.

What is MARIE?

Architecture

Machine Architecture that is Really Intuitive and Easy

Word size
16 bits, 2's complement

Memory
4096 × 16-bit words, byte-addressed by 12-bit MAR

Registers
AC (16) · MBR (16) · IR (16) · MAR (12) · PC (12) · InREG · OutREG

Instruction format
`[ 4-bit opcode | 12-bit address ]`

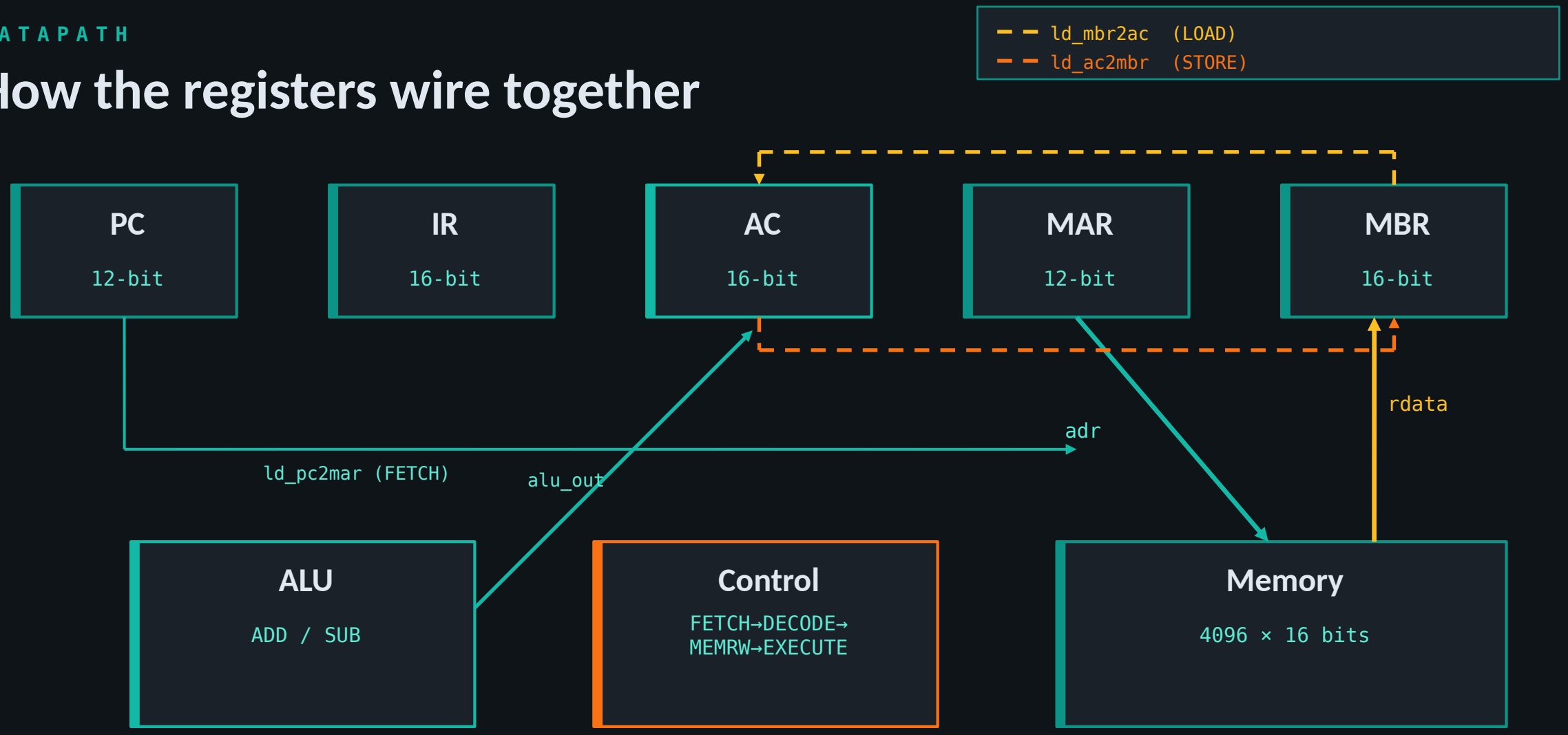
Implementation
Simple accumulator-based ISA, ideal as a teaching machine

Instruction set (16 bits each)

0	NOP	no operation
1	LOAD X	AC ← M[X]
2	STORE X	M[X] ← AC
3	ADD X	AC ← AC + M[X]
4	SUBT X	AC ← AC − M[X]
5	INPUT	AC ← InREG
6	OUTPUT	OutREG ← AC
7	HALT	stop the CPU
8	SKIPCOND	skip if cond on AC
9	JUMP X	PC ← X

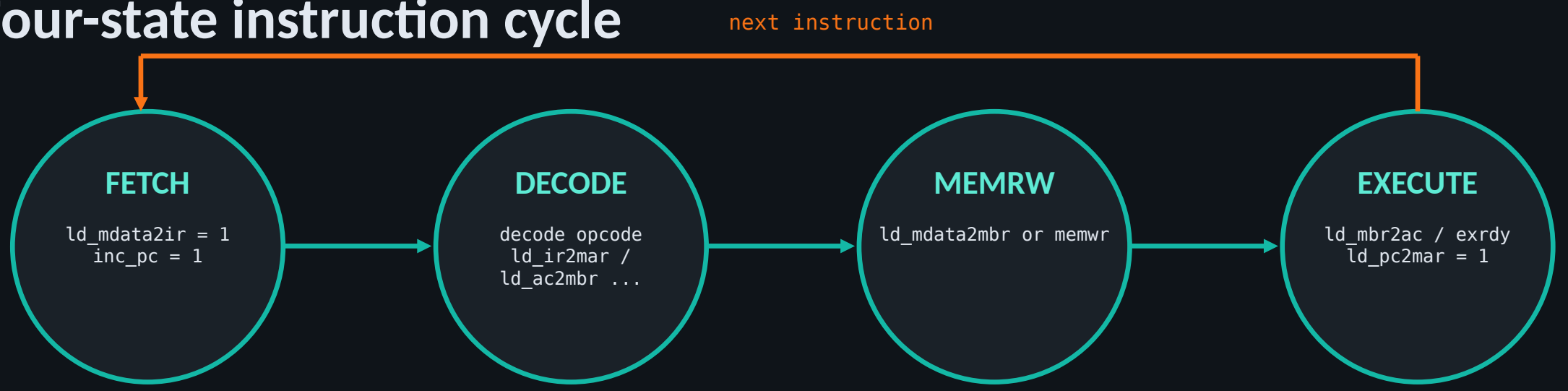
DATAPATH

How the registers wire together



Notice: MAR addresses memory. MBR holds the word in flight. AC is the only general-purpose register; every arithmetic step routes through it.

Four-state instruction cycle



Read instruction at M[PC] into IR; advance PC.

Read ir[15:12]; latch operand-address into MAR.

Read M[ir.addr] into MBR for ALU ops, or write AC to mem for STORE.

Update AC; load next-fetch address (PC) into MAR.

Side effects from DECODE

`assert_halt`
FSM freezes — clock keeps running but cpu_state stays put.

`assert_skip`
PC += 1 next clock — Skipcond evaluates ir[11:10] vs AC sign/zero.

`assert_jump`
PC ← ir[11:0] next clock — unconditional branch.

PART 1.1

The C source — small but instructive

test.c

```
// place i @ mem[16], a[0] @ mem[40]
int i = 0;
while ( i < N ) {
    a[i] = i;
    i = i + 1;
}
```

Translation challenges

No >, <, == comparisons

Express i < N as Subt N → Skipcond 000.

No while/for construct

Build it from Skipcond + two Jumps: fall-through to body, jump back at end.

No indexed addressing in this CPU

Only direct addressing is implemented (no LoadI/StoreI). PTR is updated as a value, demonstrating loop control flow.

End-of-sim check

`wait(cpu.ctrl.assert_halt)` → $display("memarray[40] = ..."); → $finish.

PART 1.1

Filled-in test.asm

```
        ORG 0

START,  Load  ZERO
        Store I
        Load  BASE
        Store PTR

LOOP,   Load  I
        Subt  N
        Skipcond 000
        Jump  END
        Jump  BODY

BODY,   Load  N
        Subt  I
        Store PTR
        Load  PTR
        Add   ONE
        Store PTR
        Load  I
        Add   ONE
        Store I
        Jump  LOOP
END,    Halt
```

Data section

```
I,    DEC 0
N,    DEC 5
ONE,  DEC 1
ZERO, DEC 0
BASE, DEC 40
PTR,  DEC 0
```

Section legend

init

Set I=0, PTR=BASE

loop

Test i < N via Subt+Skipcond

body

Update PTR and i, then re-test

halt

End the program

PART 1.2

Encoding test.asm → test.hex

Encoding rule

Each instruction = 1 word = 4 hex digits.
Top nibble = opcode, bottom 3 nibbles = address operand.

1	0	1	9
OP	addr[11:8]	addr[7:4]	addr[3:0]

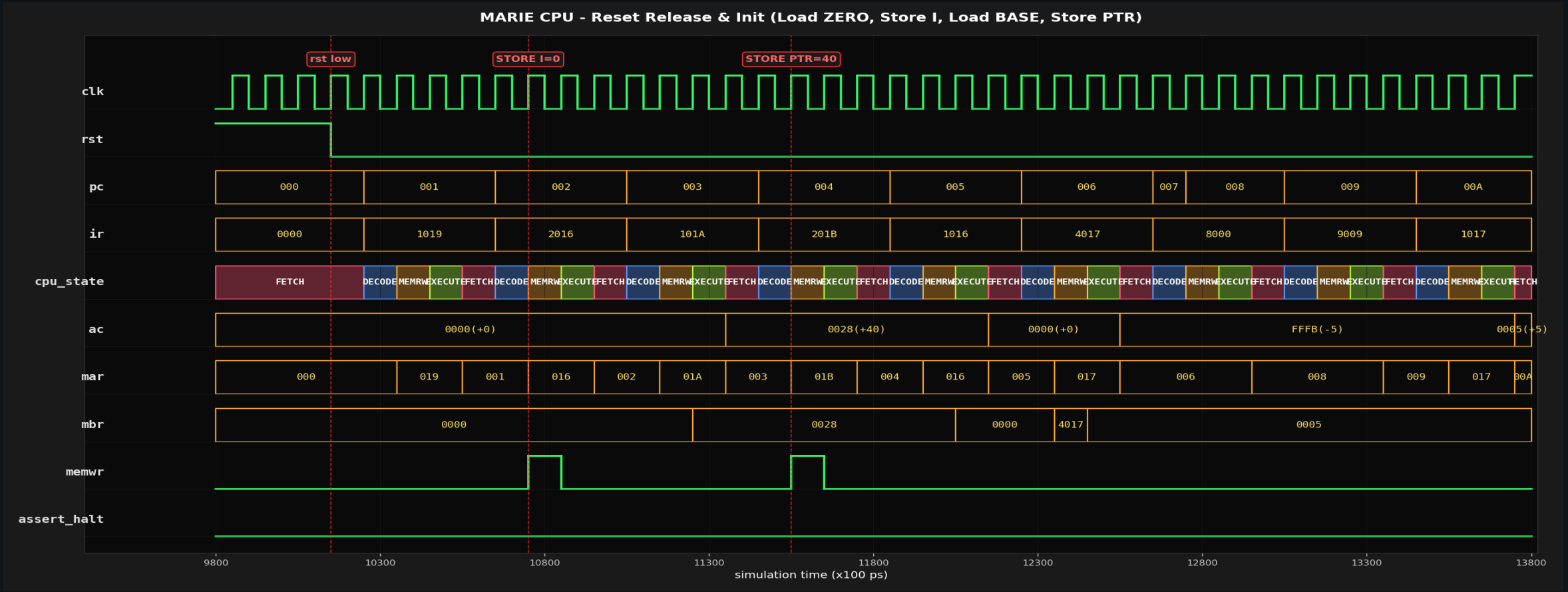
Example: Load ZERO → 0x1019 → LOAD addr 0x019 (=25 dec)

test.hex (memory image)

```
0x00    1019      Load ZERO
0x01    2016      Store I
0x02    101A      Load BASE
0x03    201B      Store PTR
0x04    1016      Load I    ; LOOP:
0x05    4017      Subt N
0x06    8000      Skipcond 000
0x07    9013      Jump END
0x08    9009      Jump BODY
0x09    1017      Load N    ; BODY:
0x0A    4016      Subt I
0x0B    201B      Store PTR
0x0C    101B      Load PTR
0x0D    3018      Add ONE
0x0E    201B      Store PTR
0x0F    1016      Load I
0x10    3018      Add ONE
0x11    2016      Store I
0x12    9004      Jump LOOP
0x13    7000      Halt        ; END:
0x14    0000      (pad)
0x15    0000      (pad)
0x16    0000      I    = 0
0x17    0005      N    = 5
0x18    0001      ONE  = 1
0x19    0000      ZERO = 0
0x1A    0028      BASE = 40
0x1B    0000      PTR  = 0
```

PART 1.3

Simulation ran clean — overview waveform

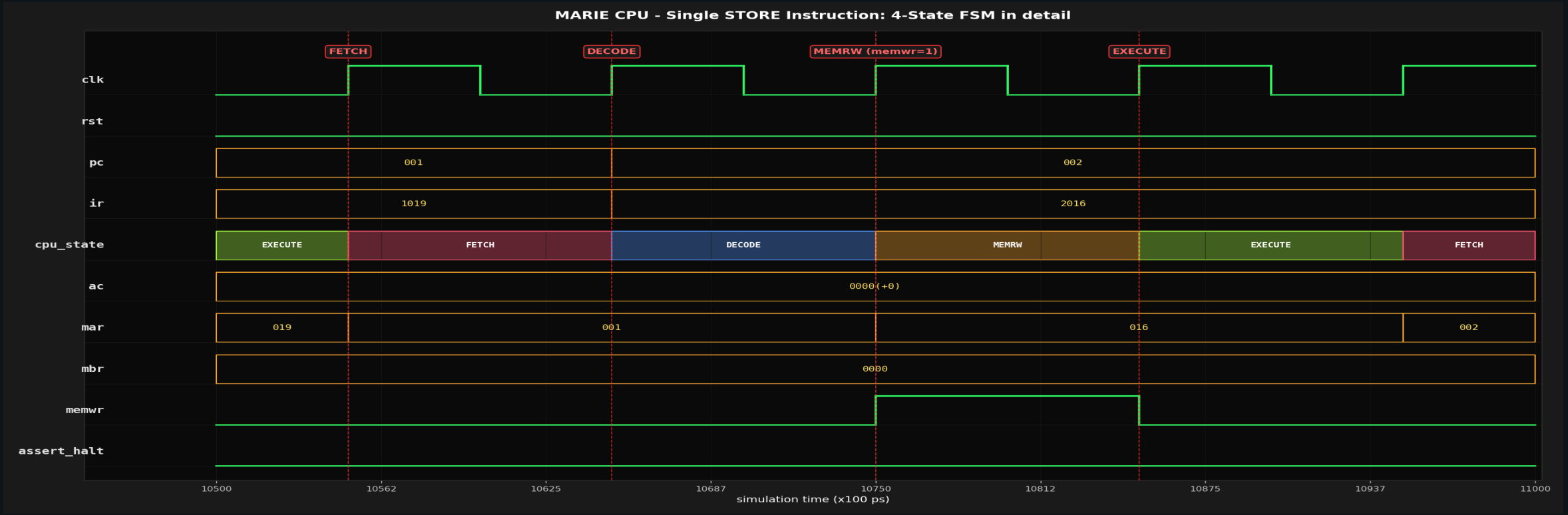



```
$ iverilog -f cpu.f && vvp a.out
```

Result: assert_halt fires at t = 41 450 → simulation done.

PART 1.3

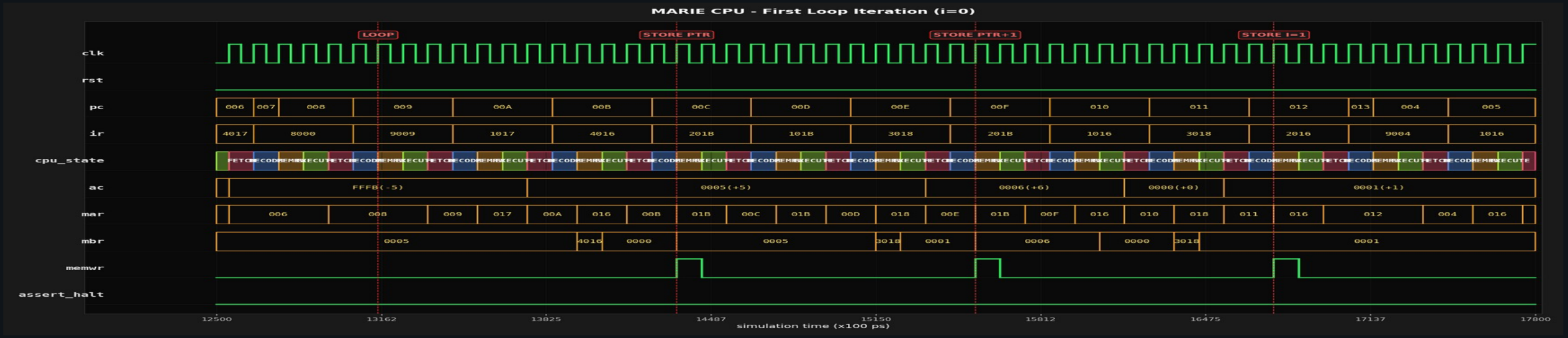
FSM in detail — one STORE instruction

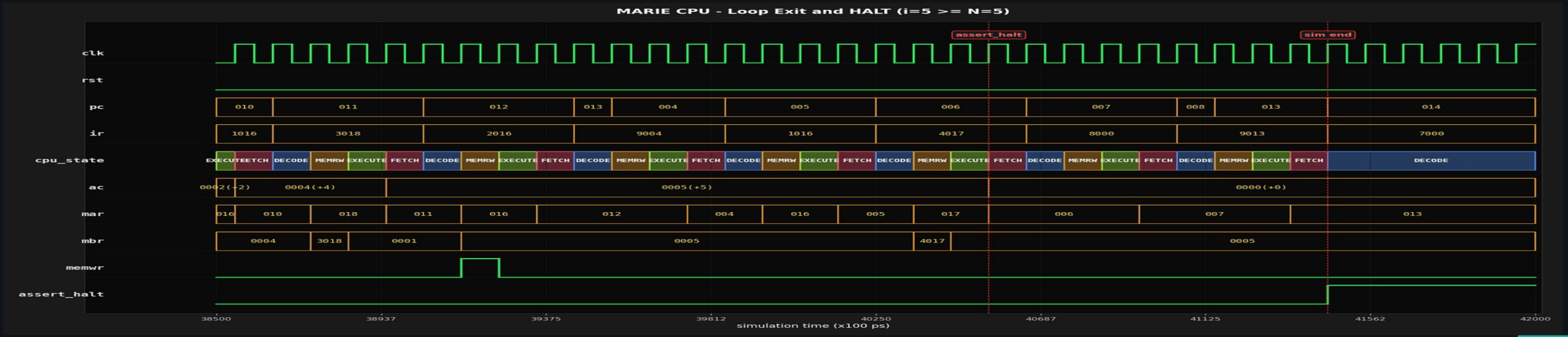


Observe: ir = 2016 → opcode 2 = STORE, address 0x16 = I.
FSM: FETCH (load IR) → DECODE (mar ← 0x016, mbr ← AC) → MEMRW (memwr=1, write happens) → EXECUTE (mar ← PC for next fetch).

PART 1.3

Loop running, then HALT asserts cleanly





Top: loop iteration #1 — three memwr pulses (Store PTR, Store PTR+1, Store I=1). Bottom: i becomes 5, condition fails, Jump END → Halt fetched, assert_halt rises.

PART 2

Issues encountered + what I took away

Issues during development

1. Skipcond polarity

Subt N gives i − N. To loop while i < N we need AC < 0 → Skipcond 000 (acneg). Mixing this up jumps to END on the first cycle.

2. Memory layout matched the sample

The provided test.hex pinned ZERO to 0x19 and I to 0x16, so I added two NOP padding words after Halt to align the data section.

3. No indirect addressing in this CPU

Only direct LOAD / STORE are wired up — no LoadI / StoreI. So a[i] = i cannot literally write to mem[40+i]. The program demonstrates the loop control flow on the PTR variable instead.

4. Reading the waveform took practice

MAR transitions twice per instruction (operand → PC). Sampling memwr at the right edge is what shows the true store address.

What I learned

Fetch / decode / execute is not abstract

Watching cpu_state cycle FETCH→DECODE→MEMRW→EXECUTE makes every diagram in Ch. 4 click. PC, MAR, MBR all earn their place on the bus.

ISA design choices have consequences

Removing indirect addressing makes a 4-line C loop hard to translate. Real architectures provide indexing because programs need it.

Verilog + iverilog is approachable

iverilog → vvp → gtkwave is one short pipeline. The same flow scales to debugging real RTL on real silicon projects.

Reading waveforms is a skill, not a feature

Knowing which signal to put on the screen — and at what zoom — is half the job. Naming things well in the testbench is the other half.

If I could redesign MARIE...

Working on this CPU made the gaps in MARIE concrete. *Five concrete improvements I would propose, in roughly cost-vs-benefit order:*

01 Indirect addressing

Add LoadI X / StoreI X using one extra control state. This is what makes a[i] = i expressible at all — without it, arrays cannot be written generically.

02 Compare instruction

A dedicated CMP X that sets condition flags would replace the Subt + Skipcond pattern. Cleaner, no AC clobber, and matches every real ISA.

03 More registers

Adding R0–R3 turns the accumulator bottleneck into a small register file. Loop counters and pointers stop fighting for AC.

04 Conditional branches

Replace Skipcond + Jump with BEQ / BNE / BLT taking an address. One instruction instead of two; control flow becomes much easier to read.

05 Stack + Call/Ret

A stack pointer and Call/Ret pair would unlock subroutines. With only Jump, every reusable block has to be inlined or hand-laid-out.